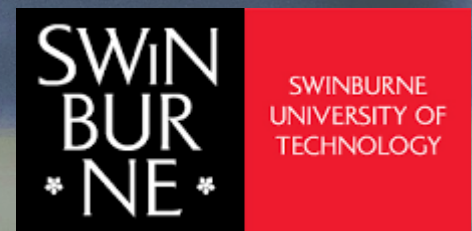




COS20028 – BIG DATA ARCHITECTURE AND APPLICATION LAB WEEK 04

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 4



ToolRunner



Combiner



USING TOOLRUNNER AND PASSING PARAMETERS

Files and Directories Used in this Exercise

Eclipse project: `toolrunner`

Java files:

`AverageReducer.java` (Reducer from `AverageWordLength`)

`LetterMapper.java` (Mapper from `AverageWordLength`)

`AvgWordLength.java` (driver from `AverageWordLength`)

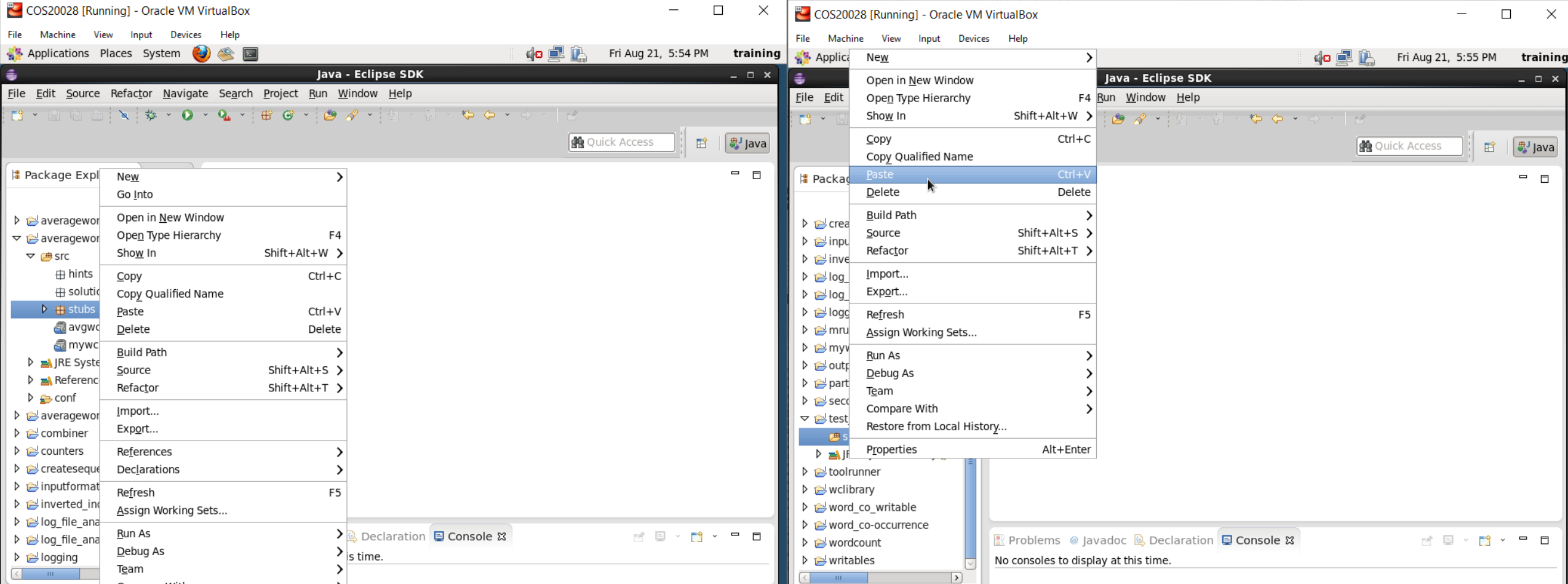
Exercise directory: `~/workspace/toolrunner`

Description

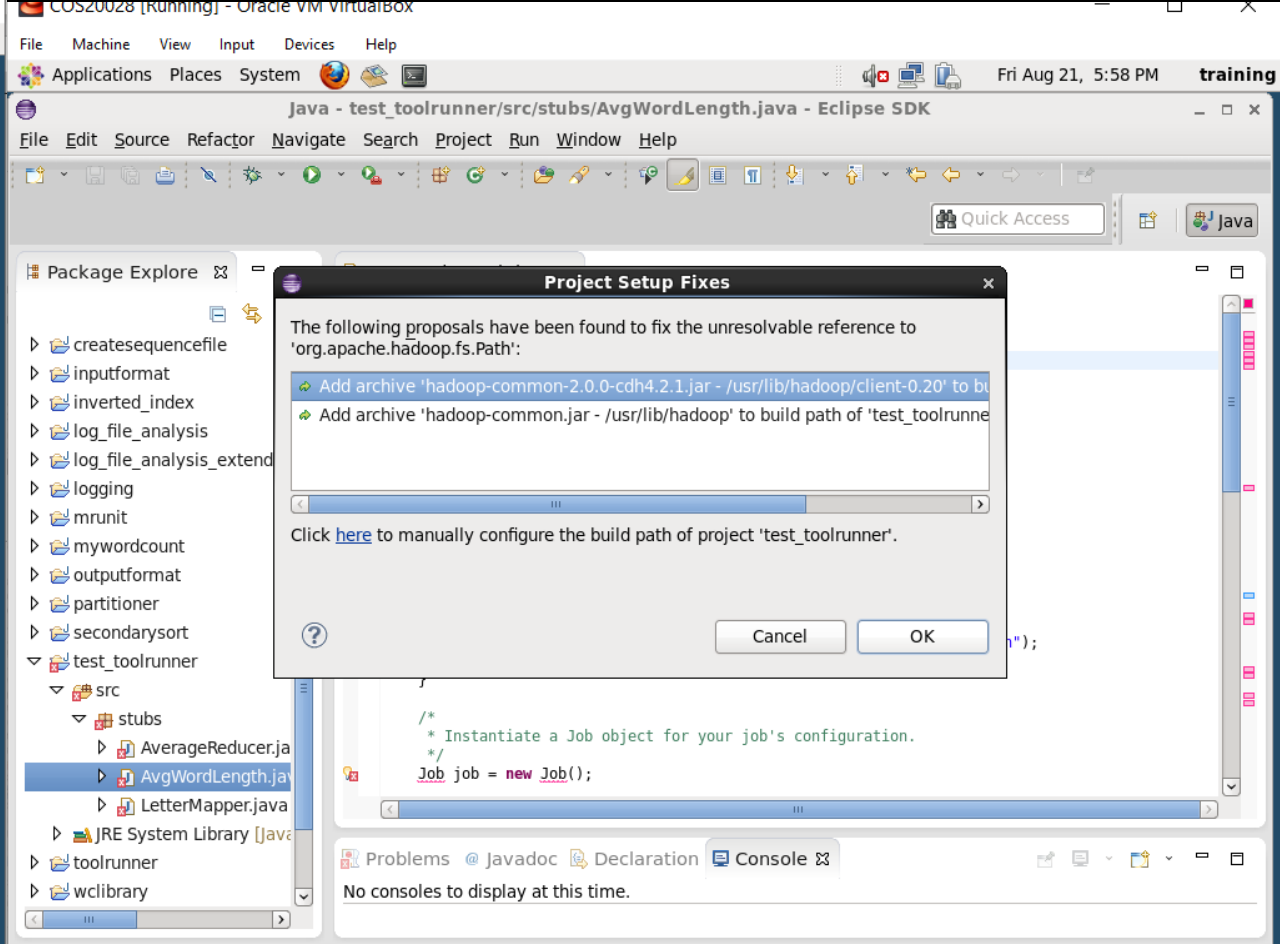
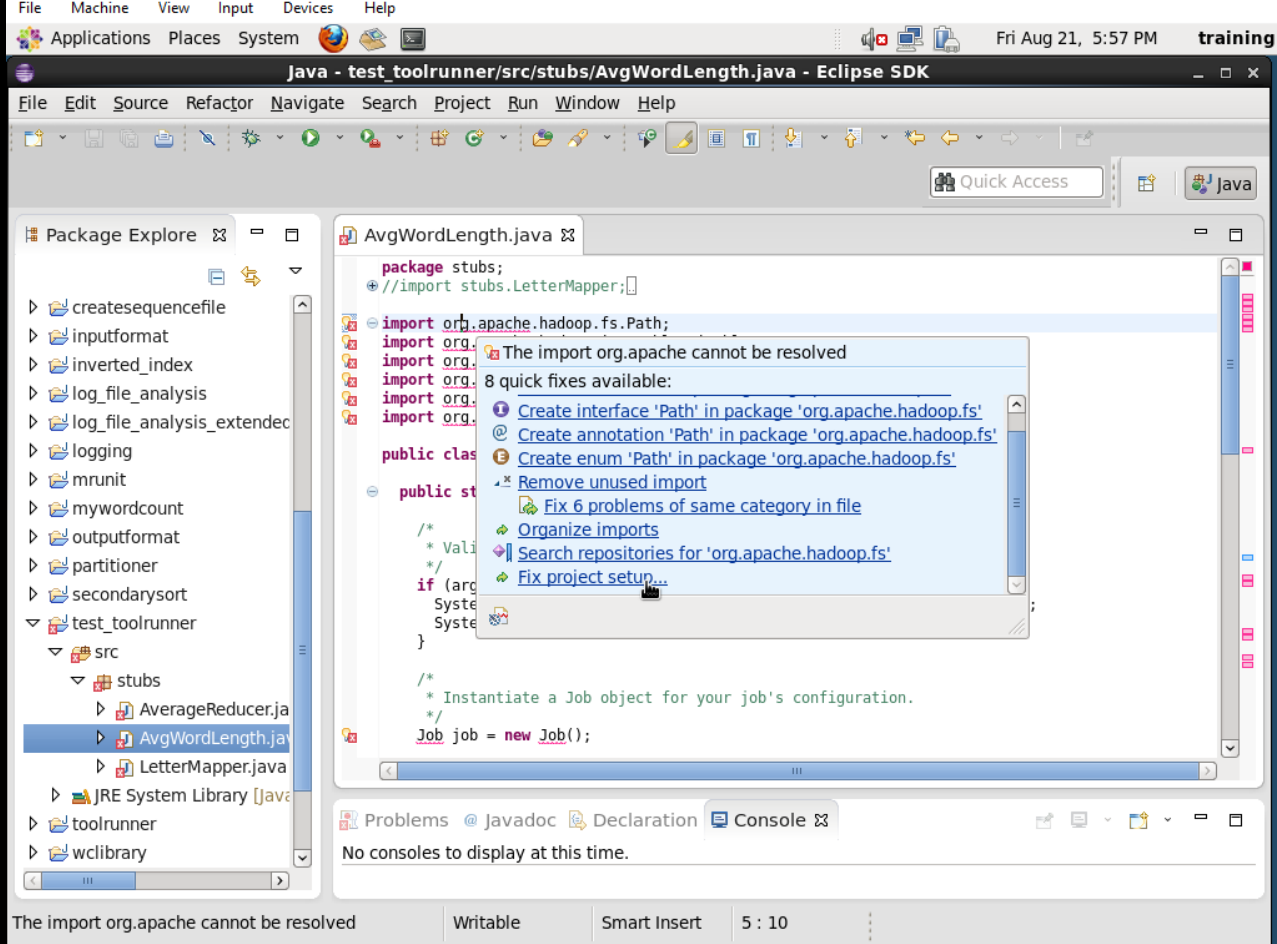
- In the first part of week 04 lab, you'll implement a driver using ToolRunner.
- Following the steps below to start with the WordCount program you wrote in an earlier lab, and modify the driver to use ToolRunner.
- Then, modify the Mapper to reference a Boolean parameter called *caseSensitive*; if true, the mapper should treat upper and lower case letters as different; if false or unset, all letters should be converted to lower case.

Copying Source Files

- Copy the Reducer, Mapper and Driver codes you completed in week 02 lab earlier in the “wordcount” project.
- You can use Eclipse to copy a Java source file from one project/package to another by right-clicking on the file and selecting Copy, then right-clicking the new package and select Past.
- If the packages have different name (i.e., if you copy from *wordcount.stubs* to *toolrunner.stubs*), Eclipse will automatically change the *package* directive at the top of the file.
- If you copy the file using a file browser or the shell, you will have to do that manually.



COPYING FILES INTO A NEW PROJECT



FIX THE IMPORT LIBRARIES AND PATHS

Modify the Driver Code and Test

Refer to Week04 lecture slides and modify the Driver code to:

- Implement the *run* method.
- Modify *main* to call *run*.

Compile your solution and test it before continuing. It should continue to function exactly as it did before.

Refer to the Week 02 lab to compile, build, submit the job, and retrieve results.

Modify the Mapper to use a Configuration Parameter

Modify the *LetterMapper* class to:

- Override the *setup* method to get the value of a configuration parameter called *caseSensitive*, and use it to set a member variable indicating whether to do case sensitive or case insensitive processing.
- In the *map* method, choose whether to do case sensitive processing (leave the letters as-is), or insensitive processing (convert all letters to lower-case) based on that variable.

Pass a Parameter Programmatically

- Modify the driver's *run* method to set a Boolean configuration parameter called *caseSensitive*. (Hint: Use the *Configuration.setBoolean* method.)
- Test your code twice: once passing *false* and once passing *true*. When set to *true*, your final output should have both upper and lower case letters; when *false*, it should have only lower case letters. (Hint: Remember to rebuild your Jar file to test changes to your code. Also, delete the output folder or use different name for the output folder when testing twice.)

Passing a Parameter as a Runtime Parameter

- Comment out the code that sets the parameter programmatically. (If you are using Eclipse, you can select the code to comment and then select Source → Toggle Comment.) Test again, this time passing the parameter value using *-D* on the Hadoop command line.

```
hadoop jar toolrunner.jar stubs.AvgWordLength -D caseSensitive=true shakespeare toolrunneroutput
```

- Test passing both *true* and *false* to confirm the parameter works correctly.



USING A COMBINER

Files and Directories Used in this Exercise

Eclipse project: `combiner`

Java files:

`WordCountDriver.java` (Driver from WordCount)

`WordMapper.java` (Mapper from WordCount)

`SumReducer.java` (Reducer from WordCount)

Exercise directory: `~/workspace/combiner`

Description

- In the second part of week 04 lab, you'll add a *Combiner* to the WordCount program to reduce the amount of intermediate data sent from the Mapper to the Reducer.
- Summing is associative and commutative and thus the same class can be used for both the Reducer and the Combiner.

Implement a Combiner

1. Copy *WordMapper.java* and *SumReducer.java* from the *wordcount* project to the *combiner* project.
 2. Modify the *WordCountDriver.java* code to add a *Combiner* for the WordCount program.
 3. Assemble and test your solution. (The output should remain identical to the WordCount application without a combiner.)
- Refer to the week 04 lecture slides (p. 25) for adding a Combiner in the code.